

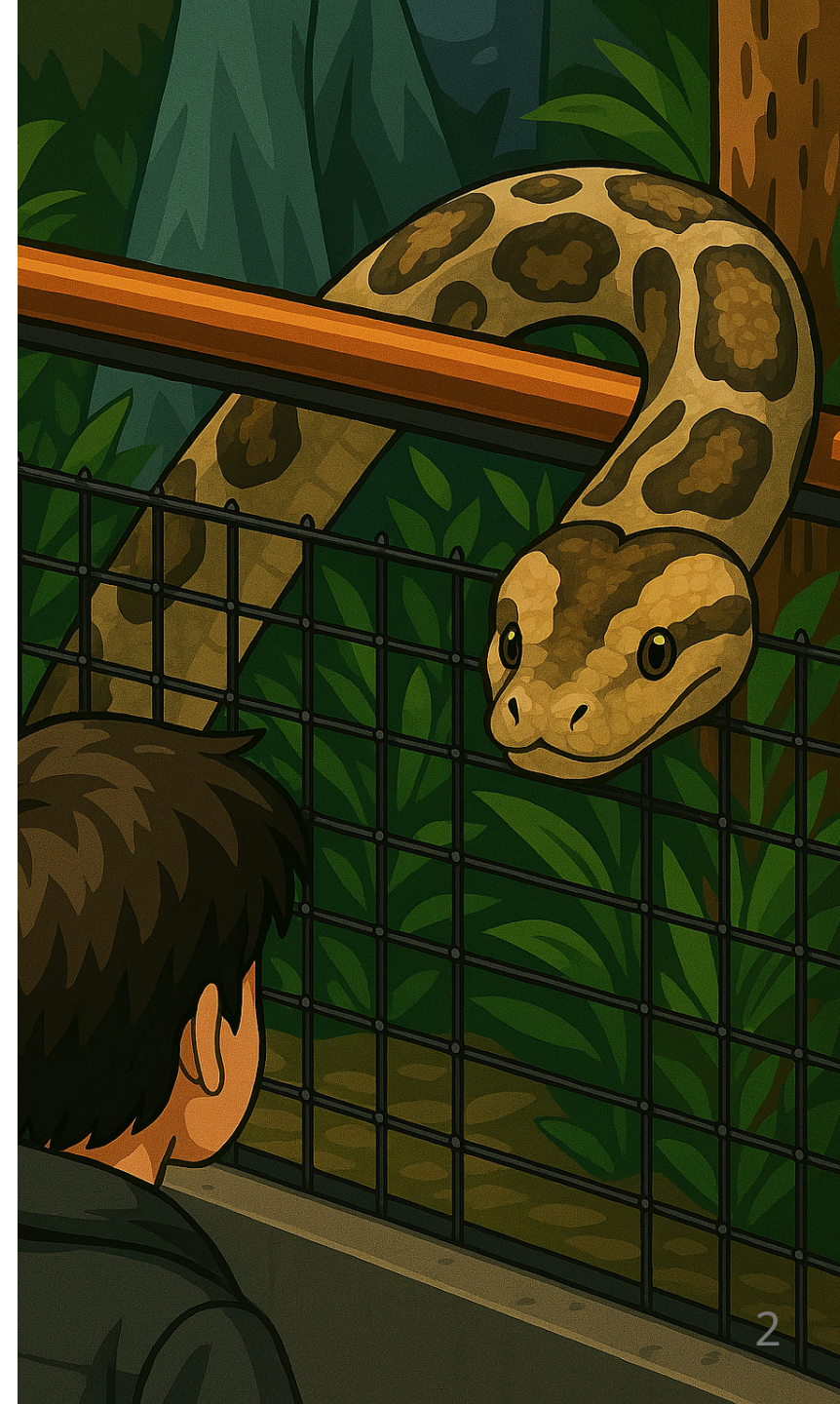
Python - Gestion des exceptions

BTS CIEL



Sommaire

- Qu'est-ce qu'une exception ?
- Comment font les autres ?
- Exceptions en Python
 - Mot clé `raise`
 - Stacktrace
 - Structure `try / except`
 - Structure `else / finally` et `with`
 - Exceptions de la bibliothèque standard
 - Exceptions sur-mesures
- Bien gérer les exceptions



Qu'est-ce qu'une exception ?

Définition

En programmation, une exception est un **évènement inattendu** (non-souhaité) qui a lieu lors de l'exécution d'une instruction.

Lorsqu'une exception a lieu, il est généralement **préférable de stopper l'exécution du programme**, mais, dans certains cas il est possible de **proposer une alternative** et de faire fonctionner l'opération autrement.

Qu'est-ce qu'une exception ?

Exemples d'exceptions

Liées au programme lui-même :

- Manque de validation (ensemble incorrect)
- Mauvais usage d'une méthode / fonction
- Opération impossible (division par zéro etc.)

Liées à l'environnement d'exécution :

- Traiter un fichier dans un format incorrect (JSON par exemple)
- Écrire dans un fichier alors que le disque dur est plein
- Communiquer sur le réseau en hors ligne

Qu'est-ce qu'une exception ?

Gestion intégrée

Les langages de haut niveau, comme Python, intègrent des **mécanismes de gestion des exceptions**. Ils permettent aux développeurs de gérer les erreurs de manière structurée et naturelle, en les intégrant comme une composante essentielle du développement.

Python propose l'utilisation de la structure `try` / `except` :

```
try:  
    x = int(input("Please enter a number: "))  
except ValueError:  
    print("Oops! That was no valid number. Try again...")
```

Comment font les autres ?

```
int main() {  
    FILE *f = fopen("filename.ext", "r");  
  
    if (f == NULL) {  
        perror("Erreur");  
        exit(EXIT_FAILURE);  
    }  
  
    return 0;  
}
```

```
try:  
    f = open("filename.ext")  
except Exception as err:  
    raise SystemExit(err)
```

```
try {  
    $f = fopen("filename.ext", "r");  
    if ($f === false) {  
        throw new Exception("Impossible d'ouvrir le fichier");  
    }  
} catch (Exception $e) {  
    die("Erreur: " . $e->getMessage());  
}
```

```
f, err := os.Open("filename.ext")  
if err != nil {  
    log.Fatal(err)  
}
```

Exceptions en Python

Une exception est une instance de la classe `BaseException`.

Les exceptions qui héritent de `BaseException` sont divisées en deux familles (héritage):

- `Exception` généralement conçues pour être traitées (catch)
- les autres (qui héritent directement de `BaseException`) comme `KeyboardInterrupt` qui ne sont pas faites pour être gérées

Exceptions en Python

Mot clé `raise`

Le mot clé `raise` permet de lever une exception.

En levant (émettant) une exception le programme bascule en **mode exception** jusqu'à ce que l'exception soit "attrapée".

Si l'exception n'est **pas attrapée**, le programme **est terminé en erreur**.

Rappel sur la sortie d'un programme

Pour rappel, en informatique un programme peut se terminer **en succès** ou **en échec**.

Un programme indique son état, à l'OS, à l'aide d'un **code de sortie**.

Généralement, **0 indique une sortie sans erreur**.

■ Ce fonctionnement est valable quelque soit le langage de programmation utilisé.

Exceptions en Python

Mot clé `raise`

Il est possible d'utiliser le mot clé `raise` depuis n'importe quel endroit du programme.

Il est utilisé pour indiquer que l'opération ne se passe pas comme prévu.

```
raise Exception("voilà une exception")
```

Dans une méthode/fonction :

```
def est_positif(n: str):  
    if n.isnumeric():  
        return int(n) > 0  
    raise Exception("n doit être une chaîne représentant un entier.")
```

Exceptions en Python

Stacktrace (traceback)

Lorsqu'une exception est levée, l'interpréteur Python **enregistre les fonctions** qui ont été traversées pour arriver jusqu'à l'exception.

Une fois le programme terminé, cet enregistrement **est écrit sur la sortie standard d'erreur** (stderr) pour vous permettre de mieux comprendre d'où vient l'exception.

Cet enregistrement se nomme **stacktrace** (ou bien **traceback** en Python).

Exceptions en Python

Stacktrace (traceback)

```
def est_positif(n: str):  
    if n.isnumeric():  
        return n > 0  
    else:  
        raise Exception("n doit être un entier.")  
  
if __name__ == "__main__":  
    est_positif("t")
```

```
Traceback (most recent call last):  
  File "/tmp/est_positif.py", line 7, in <module>  
    est_positif("t")  
  File "/tmp/est_positif.py", line 4, in est_positif  
    raise Exception("n doit être un entier.")  
Exception: n doit être un entier.
```

Exceptions en Python

Stacktrace (traceback)

```
def division(a, b):  
    return a / b  
  
def calcul():  
    return division(10, 0)  
  
if __name__ == "__main__":  
    calcul()
```

```
Traceback (most recent call last):  
  File "/tmp/calcul_exception.py", line 8, in <module>  
    calcul()  
  File "/tmp/calcul_exception.py", line 5, in calcul  
    return division(10, 0)  
          ^^^^^^^^^^^^^^^^^  
  File "/tmp/calcul_exception.py", line 2, in division  
    return a / b  
          ~~^~~  
ZeroDivisionError: division by zero
```

Structure `try` / `except`

La structure `try` / `except` permet d'attraper une exception.

Elle doit être placée sur une portion de code bien précise dans laquelle une exception est susceptible de survenir **et que l'on souhaite traiter le cas d'exception** (c.-à-d proposer un chemin alternatif ou **spécialiser** l'exception).

Certains langages comme Java obligent le développeur à traiter les cas d'exception ou préciser explicitement qu'ils sont ignorés. En Python il n'y a aucune obligation.

Structure `try` / `except`

```
import argparse

def division(a, b):
    return a / b

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("a", type=float)
    parser.add_argument("b", type=float)
    args = parser.parse_args()
    a, b = args.a, args.b

    try:
        result = division(a, b)
    except ZeroDivisionError:
        result = 0
        print("Attention : division par zéro détectée. Résultat forcé à 0.")

    print(f"Résultat : {result}")
```

Structure `try / except`

```
import argparse

def division(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        return 0

if __name__ == "__main__":
    # ...
    try:
        # L'exception ne sera jamais interceptée ici car déjà gérée dans la fonction division
        result = division(a, b)
    except ZeroDivisionError:
        result = 10 # Valeur par défaut

    print(f"Résultat : {result}")
```

Il est souvent préférable de laisser l'exception se propager pour ne pas masquer des erreurs dans d'autres parties du programme. Si on ne sait pas comment traiter une erreur, il est généralement préférable de ne pas l'intercepter.

Structure `try` / `except`

Gérer plusieurs exceptions

```
import sys

try:
    f = open('myfile.txt')
    s = f.readline()
    i = int(s.strip())
except OSError as err:
    print("OS error:", err)
except ValueError:
    print("Could not convert data to an integer.")
except Exception as err:
    print(f"Unexpected {err=}, {type(err)=}")
    raise
```

Structure `try` / `except`

Utiliser une exception comme un cas alternatif

Parfois, il est plus simple de tenter une opération et de traiter l'exception comme un **cas alternatif** plutôt que de valider les données en amont.

Attention : utiliser ce mécanisme uniquement si vous n'avez pas d'autres choix

```
def is_number(s):  
    try:  
        float(s)  
    except ValueError:  
        return False  
    else:  
        return True
```

Dans cet exemple, le code présent dans le `else` sera uniquement joué si aucune exception n'est levée.

Structure `try` \ `finally`

Le mot clé `finally` permet d'exécuter un ensemble d'instructions, à la fin du bloc, qu'une exception soit levée ou non.

```
def divide(x, y):  
    try:  
        result = x / y  
    except ZeroDivisionError:  
        print("division by zero!")  
    finally:  
        print("executing finally clause")
```

Structure `try \ finally`

Autres exemples

```
def bool_return():  
    try:  
        return True  
    finally:  
        return False
```

Structure `try \ finally`

Autres exemples

```
fichier = None
try:
    fichier = open('fichier.txt', 'r')
    contenu = fichier.read()
except Exception as e:
    print(f"Une erreur est survenue : {e}")
finally:
    if fichier is not None:
        fichier.close()
```

La clause `finally` comporte généralement des instructions de nettoyage `clean-up actions`.

Ce sont des instructions que l'on souhaite exécuter dans tous les cas.

Structure `with`

Certaines actions de nettoyage ou de finalisation ont été standardisées, et l'instruction `with` permet d'en garantir l'exécution.

Le code précédent peut (et doit) simplement s'écrire :

```
with open('fichier.txt', 'r') as fichier:  
    contenu = fichier.read()
```


Exceptions de la bibliothèque standard

Exception	Description
ValueError	Valeur incorrecte (ex: <code>int("abc")</code> , <code>math.sqrt(-1)</code>).
TypeError	Type inapproprié (ex: <code>"2" + 2</code> , <code>len(42)</code>).
IndexError	Accès à un index invalide (ex: <code>liste[10]</code> pour une liste de taille 5).
KeyError	Clé inexistante dans un dictionnaire (ex: <code>dico["inexistante"]</code>).
FileNotFoundError	Fichier introuvable (ex: <code>open("fichier_inexistant.txt")</code>).
IOError	Erreur d'entrée/sortie (ex: disque plein, permission refusée).
AttributeError	Attribut ou méthode inexistant (ex: <code>"str".append("x")</code>).
ImportError	Module ou symbole introuvable (ex: <code>import module_inexistant</code>).
KeyboardInterrupt	Interruption par l'utilisateur (Ctrl+C).
OSError	Erreur liée au système d'exploitation (ex: chemin invalide).
RuntimeError	Erreur générique à l'exécution (rarement levée directement).

Exceptions sur-mesures

Une exception est une **instance** de la classe `BaseException` .

En utilisant le principe d'héritage, il est possible de créer ses propres classes d'exceptions pour communiquer des erreurs ou des comportements anormaux aux autres développeurs.

■ Nous verrons cela en détail avec le cours sur la programmation orientée objet.

Bien gérer les exceptions

Quelques questions à se poser avant d'utiliser le mécanisme d'exception :

- Est-ce qu'un pattern `with` standard est associé avec l'objet / méthode que j'utilise ?
- Est-ce que la méthode/fonction appelée est susceptible de générer des exceptions (voir la documentation) ?
- Suis-je capable de traiter le cas d'exception ?
- Est-ce nécessaire de donner plus d'informations à l'utilisateur (faut-il spécialiser l'exception) ?
- Ais-je une autre option si je souhaite représenter un cas alternatif ?